NegativeHours: Milestone 2 summary

Corsin Hidber · HSLU MLOps HS26 · Repo: `CorsinAI/MLOPS-Project-HSLU-HS-26`

1. Summary of the milestone

What a reviewer should look at first

Start with `docs/ms2_summary.md`, then these key files for each step of the feature pipeline:

1. **Run and schedule:** `src/neghour/pipelines/feature_pipeline.py`, `.github/workflows/daily_pipeline.yml`
2. **Fetch:** `src/neghour/sources/smard.py`, `src/neghour/sources/open_meteo.py`
3. **Publication rules and time:** `src/neghour/availability.py`, `src/neghour/timeutils.py`
4. **Build features and labels:** `src/neghour/features/build.py`, `src/neghour/features/calendar.py`
5. **Validate:** `src/neghour/features/validation.py`, `tests/test_leakage.py`
6. **Store:** `src/neghour/storage/hopsworks_store.py`, `scripts/check_store_quality.py`

What works now

The feature pipeline runs end to end. Every day, GitHub Actions collects prices and power data from SMARD and weather forecasts from Open-Meteo. It builds one row per delivery hour, using only what was known at 10:00 Berlin on the day before, checks the rows and saves them to Hopsworks. Later runs add the real prices as labels once they are published. Live runs and backfills share the same code. Training and inference are not part of this milestone.

The pipeline has run successfully when started by hand, on its daily schedule and inside the Docker image. The stored history covers January 2024 to November 2026: about 25,000 hours, each stored once. Clock-change days have their correct 23 or 25 hours, and no row contains data from after its cut-off. The only gaps are in early 2024, before archived weather forecasts begin. About 6 % of all hours are negative, and all automated tests pass.

Key decisions

- **Label:** `y_neg` = hourly mean price < 0 from SMARD filter 4169. It stays closest to the pre-switch hourly price across the whole dataset. `n_neg_quarters` is stored so a quarter-hour label can be tested later.
- **Vintage:** `_previous_day2` forecasts in both backfill and live, so training and serving see forecasts of the same age.
- **Time:** storage in UTC, keyed by UTC hour; delivery days and the cut-off in Europe/Berlin. Rows are selected by timestamp, so a late cron run writes the same features.
- **Availability:** rule-based, because SMARD publishes no per-row timestamp.
- **Late data:** NaN plus a `missing_*` flag, never carried forward.

Known limits: GitHub cron is best effort, so a start before the cut-off is likely, not guaranteed. SMARD revisions and the stability of `_previous_day2` values between runs are not measured. The 6-hour publication margins are assumptions. One container run (41920298746) failed on a Hopsworks lock timeout, so writes now retry on server errors; that retry has only been tested with fakes. All decisions are in `docs/ms2_decisions.md`.

2. Response to reviews

Response to E1: I checked filter 4169: it stays hourly after the switch, and each value is the mean of the four quarters. I use mean < 0 as the label, because it stays closest to the pre-switch hourly price. An "any quarter" label would suit a battery operator better, but it would raise the share of negative hours at the switch, so training and test would measure different things. The cost is 116 actionable hours missed since the switch, so I store `n_neg_quarters` for MS3 to test that label. `fetch_prices` reads the same hourly series throughout, and `fetch_negative_quarters` reads quarter data from the switch onwards only.
Status: Changed. Commits 6603e83, 6f812e5, 85bbfe8. File: `src/neghour/sources/smard.py`.

Response to E2: It is a classification task: will an hour have a negative price? I do not predict exact prices, because the decision is yes or no per hour. Prices, production and load need no regional treatment, because SMARD publishes them for the whole DE-LU zone. Weather varies by location, so I average 4 fixed points per variable. Wind uses the north coast, north-east and centre, where most German wind capacity is. Cloud cover and irradiance use the north-east, centre and south, where large solar parks and rooftop solar are most common. Temperature uses all 4, because it drives demand everywhere. The points are an educated guess and might need adjustment.
Status: Changed. Commits 6f812e5, 304cb25, 72e3906. Files: `docs/proposal_addendum.md`, `src/neghour/config.py`, `src/neghour/sources/open_meteo.py`.

Response to E2 (seasonality): Time of day, day of week and time of year are sine and cosine features, so a model can see that hour 23 is next to hour 0. `is_weekend` covers the lower demand at weekends. How the model uses them follows in MS3.
Status: Partly. Commit c0d414d. File: `src/neghour/features/calendar.py`.

Response to E2 (upsampling): I do not upsample, because that would change the class balance in training but not in serving. MS3 will use `scale_pos_weight` instead.
Status: Partly. Commit 021b736. File: `README.md`, section "Class imbalance".

Response to A1: I use `_previous_day2` in both backfill and live, so training and serving see forecasts of the same age. `_previous_day1` is fresher, but for later hours of D it would use runs made after the cut-off. A value counts as available at T - 48 h + 6 h, and the 6-hour margin is an assumption.
Status: Changed. Commit 304cb25. File: `src/neghour/sources/open_meteo.py`.

Response to A3: The cut-off is a rule in code, checked in three places. Every run stops before writing if a row has data from after the cut-off. The tests change or remove later data and require identical rows. A report rechecks the full stored table, which has 0 late rows.
Status: Changed. Commits 245c48b, aa75bc7, 689eb2f. Files: `src/neghour/features/validation.py`, `tests/test_leakage.py`, `scripts/check_store_quality.py`.

Response to B1: Morning runs are scheduled at 01:17 and 05:17 UTC, because cron started this repo's jobs 4 to 9 hours late. The first scheduled run started at 08:56 Berlin. A start before 10:00 is likely, not guaranteed, but a late run writes the same features, because rows are selected by timestamp. A 12:17 UTC run refreshes the labels, and the 10:00 cut-off stays. A missing or late input gives NaN plus a flag and a log line, because a copied last value would look like real data.
Status: Changed. Commits c91f563, dc8bdbf, 94863b8. Files: `.github/workflows/daily_pipeline.yml`, `src/neghour/pipelines/feature_pipeline.py`.

Response to A2: Europe/Berlin is stated throughout, and storage is in UTC. The cut-off is computed in Berlin time, so the UTC cron shift cannot move it. Tests check the cron slots in CET and CEST, and days of 23 and 25 hours are confirmed in storage.
Status: Changed. Commits 49ebba4, b9f0ddc. Files: `src/neghour/timeutils.py`, `tests/test_schedule.py`.

Response to B2: `holiday_share` is the share of Germany's population with a public holiday that day, by state. It sits next to `is_holiday`, which is 1 only when all 16 states are off. Luxembourg is about 0.8 % of the zone and is left out.
Status: Changed. Commit c0d414d. File: `src/neghour/features/calendar.py`.

Response to E2 (originality check): I couldn't find exactly what you were referring to. If you link me the papers, I can tell you for certain. The closest work I found is a KTH thesis on SE3 (Vanherpe, 2026), which I checked from its abstract. I predict whether hours will be negative, whereas it studies how weather forecast errors affect intraday prices.
Status: Partly. Commit 021b736. File: `README.md`, "Originality" paragraph.

Response to E3: The plan is written: PR-AUC and F1 over the last 30 delivery days against both baselines, plus a weekly drift check. It is not built, because it needs a model and stored predictions.
Status: Partly. Commit 1e15614. File: `README.md`, section "Monitoring plan (MS3)".

Response to E4: Since the skeleton I have made over 40 focused commits, with one pull request per phase or fix.
Status: Changed. PRs 1, 2, 4-15. File: `docs/ms2_decisions.md`, entry 1.